Binary Search Trees (BST) Mechanics and Degradation

A Binary Search Tree satisfies the invariant that for any node X, all keys in the left subtree are strictly less than X.key, and all keys in the right subtree are strictly greater than X.key. In an ideal balanced tree of height h = floor(log2(n)), search, insert, and delete take O(log n) time. However, inserting pre-sorted keys causes the BST to degenerate into an O(n) linear linked list.

Self-Balancing AVL Trees and Rotation Invariants

An AVL tree guarantees an O(log n) worst-case height by enforcing that for every node, the Balance Factor (BF = height(LeftSubtree) - height(RightSubtree)) is strictly in {-1, 0, 1}.

When an insertion causes BF to become +2 or -2, tree rotations restore balance:
1. Left-Left (LL) Heavy: Fixed via a single Right Rotation.
2. Right-Right (RR) Heavy: Fixed via a single Left Rotation.
3. Left-Right (LR) Heavy: Fixed via a Double Rotation: Left rotate on left child, followed by Right rotate on unbalanced root.
4. Right-Left (RL) Heavy: Fixed via a Double Rotation: Right rotate on right child, followed by Left rotate on unbalanced root.

Binary Heaps and Priority Queue Operations

A Binary Heap is a complete binary tree satisfying the Heap Property. In a Min-Heap, parent.key <= child.key for all nodes. Arrays efficiently represent binary heaps without pointers: for node index i, left_child = 2i + 1, right_child = 2i + 2, and parent = floor((i - 1) / 2).

Insertion (heappush): Append element to end of array and bubble-up (sift-up) in O(log n) time. Extraction (heappop): Replace root with last element and bubble-down (sift-down) in O(log n) time. Floyd's Heapify builds a valid heap in-place in O(n) linear time by sifting down nodes from floor(n/2) down to 0.